

```

public class TreeNode {
    TreeNode left;
    TreeNode right;
    int data;

    TreeNode(int data) {
        this.left = null;
        this.right = null;
        this.data = data;
    }
}

public class BinarySearchTree {
    TreeNode root;
    public boolean deleteMin = true;

    private TreeNode insertTreeNOde(TreeNode root, TreeNode new_node) {

        int data = new_node.data;
        if (root == null) {
            root = new_node;
            return root;
        }
        if (data < root.data) {
            root.left = insertTreeNOde(root.left, new_node);

        } else if (data > root.data) {
            root.right = insertTreeNOde(root.right, new_node);

        }
        return root;
    }

    public void insert(int data) {
        TreeNode new_node = new TreeNode(data);
        this.root = insertTreeNOde(this.root, new_node);
    }

    private void inorderTraversal(TreeNode root) {
        if (root != null) {
            inorderTraversal(root.left);
            System.out.print(root.data + " ");
            inorderTraversal(root.right);
        }
    }
}

```

```

public void inOrder() {
    inorderTraversal(this.root);
}

private void preorderTraversal(TreeNode root) {
    if (root != null) {
        System.out.print(root.data + " ");
        preorderTraversal(root.left);
        preorderTraversal(root.right);
    }
}

public void preOrder() {
    preorderTraversal(this.root);
}

private void postorderTraversal(TreeNode root) {
    if (root != null) {
        postorderTraversal(root.left);
        postorderTraversal(root.right);
        System.out.print(root.data + " ");
    }
}

public void postOrder() {
    postorderTraversal(this.root);
}

private TreeNode searchNode(TreeNode root, int target) {
    if (root == null) {
        return null;
    } else {
        if (root.data == target) {
            return root;
        } else if (target > root.data) {
            return searchNode(root.right, target);
        } else {
            return searchNode(root.left, target);
        }
    }
}

public void search(int target) {
    TreeNode temp = searchNode(root, target);
    if (temp == null) {
        System.out.println("Value not found");
    } else {

```

```

        System.out.println("value found : " + target);
    }
}

public TreeNode deleteNode(TreeNode root, int target) {
    if (root == null) {
        return root;
    }
    if (target < root.data) {
        root.left = deleteNode(root.left, target);

    } else if (target > root.data) {
        root.right = deleteNode(root.right, target);
    } else {
        if (root.left == null && root.right == null) {
            // System.out.println("Deleting : " + target);
            return null;
        } else if (root.left == null) {
            // System.out.println("Deleting : " + target);
            return root.right;
        } else if (root.right == null) {
            // System.out.println("Deleting : " + target);
            return root.left;
        } else {

            if (deleteMin == true) {
                TreeNode minRight = minVal(root.right);
                root.data = minRight.data;
                root.right = deleteNode(root.right, minRight.data);
                System.out.println("using minimum subtree");
            } else {
                TreeNode maxLeft = maxVal(root.left);
                root.data = maxLeft.data;
                root.left = deleteNode(root.left, maxLeft.data);
                System.out.println("using max subtree");
            }

        }

    }
    return root;
}

private TreeNode minVal(TreeNode root) {
    if (root.left == null) {
        return root;
    }
}

```

```

        return minVal(root.left);
    }

    private TreeNode maxVal(TreeNode root) {
        if (root.right == null) {
            return root;
        }
        return maxVal(root.right);
    }

    public void delete(int target) {
        this.root = deleteNode(this.root, target);
    }
}



---


public class Driver {

    public static void main(String[] args) {

        BinarySearchTree tree = new BinarySearchTree();

        tree.insert(18);
        tree.insert(9);
        tree.insert(26);
        tree.insert(3);
        tree.insert(16);
        tree.insert(20);
        tree.insert(34);
        tree.insert(1);
        tree.insert(5);
        tree.insert(11);
        tree.insert(17);
        tree.insert(19);
        tree.insert(23);
        tree.insert(32);
        tree.insert(13);
        tree.insert(21);
        tree.insert(12);
        tree.insert(15);

        // // i
        tree.search(1);
        tree.delete(1);
        tree.search(1);

        System.out.println(" ");
    }
}

```

```
// // ii
tree.search(34);
tree.delete(34);
tree.search(34);

System.out.println(" ");

// iii , delete method uses minimum subtree method by default

tree.delete(9);
tree.search(9);

System.out.println(" ");

// setting deleteMin variable to false to choose the maximum subtree method

tree.deleteMin = false;

tree.delete(20);
tree.search(20);

}

}
```